

Library Management System

1. Project Overview

- project designed to manage books, members, and issue/return records efficiently. The system is developed using C++ for backend logic and HTML, Tailwind CSS, and JavaScript for frontend interface. This project is focused on combining strong backend logic with an interactive and user-friendly interface. The goal is to simulate how real-world library management software works behind the scenes.

2. Project Objectives:

Apply **Object-Oriented Programming (OOP)** concepts in C++

- Design a clean and responsive UI using Tailwind CSS
- Implement dynamic features using JavaScript
- Integrate backend logic with frontend interface
- Understand real-world system flow and debugging

Book Management Module

Students must implement:

- Add new book
- Update book details
- Delete book
- View all books
- Search book (by title, author, or ID)

Each book should contain:

- Book ID
- Title
- Author
- Category
- Quantity

- Availability Status

□ Student / Member Management

Features required:

- Add new member
- Update member record
- Delete member
- View all members

Each member should contain:

- Member ID
- Name
- Department/Class
- Contact Information
- Issued Books List

Issue & Return System

Students must implement:

- Issue book to member
- Return book
- Prevent issuing if book quantity is 0
- Update book availability automatically
- Maintain issued book record

Bonus (optional):

- Add due date
- Calculate fine for late return

Search Functionality

Search system must support:

- Search book by ID
- Search book by Title
- Search member by ID
- Real-time frontend filtering using JavaScript

Technical Requirements

C++ (Backend Logic)

Students must:

- Use OOP concepts:
 - Classes
 - Objects
 - Constructors
 - Encapsulation
- Store data using:
 - Vectors or arrays
 - OR File handling (recommended)
- Implement:
 - Menu-driven system
 - Proper validation
 - Error handling

Example structure:

- Class Book
- Class Member
- Class Library (main controller class)

HTML Structure

Students must create:

- Dashboard page
- Add book form
- Member registration form
- Issue/Return form
- Book listing table

Forms must include proper:

- Labels
- Input validation
- Buttons

Tailwind CSS Design Requirements

UI must be:

- Fully responsive
- Clean and modern
- Proper spacing (padding & margin)
- Use cards for sections
- Styled tables
- Hover effects on buttons

Students must:

- Use Tailwind utility classes properly
- Avoid inline CSS
- Maintain design consistency

JavaScript Functionality

Students must implement:

- Form validation
- Dynamic table updates
- Search filtering
- Show/Hide sections
- Alert messages

Bonus:

- Modal popups
- Confirmation before delete

System Flow

1. Admin logs into dashboard
2. Admin adds books
3. Admin registers members
4. Admin issues book
5. System updates availability
6. Member returns book
7. System updates records
8. Add dark mode functionality

Students must draw a:

- Flowchart
- System architecture diagram